Lab Manual

of

Object Oriented Programming

Subject Code: 204ES

Academic Year 2026-27

Name of Student	:	Neel Parag Thakkar
Student ID No	:	25EC407
Batch	:	A
Lab Faculty Name	:	Mayur Sevak

BACHELOR OF ENGINEERING

In

Electronics and Communication



BVM Engineering College

(An Autonomous Institution)

Vallabh Vidhyanagar-388120

BVM ENGINEERING COLLEGE

(An Autonomous Institution)

Electronics and Communication Department

AY: 2026-27, Semester-III

CERTIFICATE

This is to certify that Mr. / Ms. **Neel Parag Thakkar** of Year/Sem. 2nd / III ID No 25EC407 has satisfactorily completed his/her term work in Month of _____ for the term ending in November-2026. Number of practical certified _____ out of _____ in the subject of **204ES: Object Oriented Programming**.

Date: / / 2026

Signature of Teacher



BVM ENGINEERING COLLEGE

(An Autonomous Institution)

Electronics and Communication Department

AY: 2026-27, Semester-III

Object Oriented Programming (204ES)

INDEX

Name: Neel Parag Thakkar **ID No.:** 25EC407

Pr. No.	Title	Date	Sign
1.	Write a Program to display basic details of a User like Name, Roll no, Division and Address.		
2.	Write a Program to make simple calculator using Switch case.		
3.	Write a Program to Convert number of Days into Number of Months and remaining days.		
4.	Write a Program to enter marks of 6 subject and find out Total Marks and Percentage.		
5.	Write a Program to find whether the given number is even or odd.		
6.	Write a Program to find the largest among three numbers.		
7.	Write a Program to find whether given number is Armstrong or not.		
8.	Write a Program which generates table of a given number.		
9.	Write a Program which uses concept of Reference Variable.		
10.	Write a Program to swap two numbers using Call by Value method.		
11.	Write a Program to swap two numbers using Call by Reference Method.		
12.	Write a Program which uses Function as default argument.		
13.	Write a Program to calculate factorial of a number using recursion.		
14.	Write a Program to take input in an array and print its sum.		
15.	Write a Program to read string and print total number of characters.		
16.	Write a Program to create a class called 'Student' that has Name, ID, CPI, SPI and Semester as data Member. Also uses Proper member Function for taking and Displaying information.		
17.	Write a Program to count number of objects using static variable.		
18.	Define a class to represent a bank account which includes the following members as data member: a) Name of Depositor b) Account Number c) Withdrawal Amount d) Balance. Member Function: a) To assign initial Value b) To Deposit an amount c) To withdraw an amount after checking balance d) To display name and balance. Write the above program for handling n number of account holder using array of object.		
19.	Write a Program to demonstrate the use of Constructors and Destructors.		

Pr. No.	Title	Date	Sign
20.	Write a Program which reads the complex number, copy that into other using copy constructor.		
21.	Write a Program implement function Overloading for geometric area.		
22.	Write a Program for addition of two complex number using Friend Function. (Use constructor function to initialize data member of complex class.)		
23.	Write a Program to find the mean value of a given number using friend function.		
24.	Write a program to add integer value of two class objects.		
25.	Write a Program to do addition of two times objects.		
26.	Write a Program to design a class having static member function named showcount() which has the property of displaying the number of object created of the class.		
27.	Write a program which uses concept of Unary Operator (++/–) overloading.		
28.	Write a program which uses concept of Binary Operator overloading on String.		
29.	Write a program to create a Matrix and overload the '+' operator to add two matrix object.		
30.	Define two classes polar and rectangle to represent point in polar and rectangle systems. Use conversion routines to convert from one system to the other.		
31.	Write a program to implement Single Inheritance.		
32.	Write a program to implement Multiple Inheritance.		
33.	Write a program to create File with constructor Function.		
34.	WAP to demonstrate the use of Template.		

Practical No. 1

Display basic details of a user

Aim

To write a C++ program to display basic details of a user such as Name, Roll No, Division and Address.

Algorithm

1. Start.
2. Declare variables for Name, Roll No, Division and Address.
3. Accept input from the user.
4. Display all the details.
5. Stop.

Program

```cpp
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string name, division, address;
    int roll;

    cout << "Enter Name: ";
    getline(cin, name);
    cout << "Enter Roll No: ";
    cin >> roll;
    cin.ignore();
    cout << "Enter Division: ";
    getline(cin, division);
    cout << "Enter Address: ";
    getline(cin, address);

    cout << "\n------ Student Details ------";
    cout << "\nName     : " << name;
    cout << "\nRoll No  : " << roll;
    cout << "\nDivision : " << division;
    cout << "\nAddress  : " << address;
    return 0;
}
```

Output

```
Enter Name: Neel Parag Thakkar
Enter Roll No: 407
Enter Division: EC-A
Enter Address: Vallabh Vidyanagar, Anand

------ Student Details ------
Name     : Neel Parag Thakkar
Roll No  : 407
Division : EC-A
Address  : Vallabh Vidyanagar, Anand
```

Practical No. 2

Simple calculator using switch-case

Aim

To write a C++ program to perform simple arithmetic operations (Addition, Subtraction, Multiplication, Division and Modulus) using the switch-case statement.

Theory

A switch-case statement is a multi-way decision-making statement in C++. A variable or expression is compared against several case labels and the block of the matching case is executed. Here the arithmetic operator entered by the user selects the operation to perform.

Algorithm

1. Start the program.
2. Declare two numbers and an operator.
3. Accept the two numbers from the user.
4. Accept an arithmetic operator (+, -, *, /, %).
5. Use the switch statement to perform the selected operation.
6. Display the result.
7. Stop the program.

Program

```cpp
#include <iostream>
using namespace std;

int main()
{
    int num1, num2;
    char op;

    cout << "Enter first number: ";
    cin >> num1;
    cout << "Enter second number: ";
    cin >> num2;
    cout << "Enter operator (+, -, *, /, %): ";
    cin >> op;

    switch (op)
    {
    case '+':
        cout << "Addition = " << num1 + num2;
        break;
    case '-':
        cout << "Subtraction = " << num1 - num2;
        break;
    case '*':
        cout << "Multiplication = " << num1 * num2;
        break;
    case '/':
        if (num2 != 0)
            cout << "Division = " << (float)num1 / num2;
        else
            cout << "Division by zero is not allowed.";
```

```
32          break;
33      case '%':
34          if (num2 != 0)
35              cout << "Modulus = " << num1 % num2;
36          else
37              cout << "Modulus by zero is not allowed.";
38          break;
39      default:
40          cout << "Invalid Operator!";
41      }
42      return 0;
43  }
```

Output 1

```
Enter first number: 48
Enter second number: 6
Enter operator (+, -, *, /, %): *
Multiplication = 288
```

Output 2

```
Enter first number: 7
Enter second number: 2
Enter operator (+, -, *, /, %): /
Division = 3.5
```

Output 3

```
Enter first number: 64
Enter second number: 9
Enter operator (+, -, *, /, %): -
Subtraction = 55
```

Output 4

```
Enter first number: 29
Enter second number: 0
Enter operator (+, -, *, /, %): %
Modulus by zero is not allowed.
```

Practical No. 3

Convert days into months and remaining days

Aim

To write a C++ program to convert a given number of days into equivalent months and remaining days.

Theory

It is assumed that 1 month = 30 days. Months = Total Days / 30 (integer division) and Remaining Days = Total Days % 30.

Algorithm

1. Start the program.
2. Declare integer variables days, months and remainingDays.
3. Accept the total number of days.
4. Calculate months = days / 30.
5. Calculate remainingDays = days % 30.
6. Display months and remaining days.
7. Stop the program.

Program

```cpp
#include <iostream>
using namespace std;

int main()
{
    int days, months, remainingDays;

    cout << "Enter the number of days: ";
    cin >> days;

    months = days / 30;
    remainingDays = days % 30;

    cout << "\nEquivalent Time:";
    cout << "\nMonths         = " << months;
    cout << "\nRemaining Days = " << remainingDays;
    return 0;
}
```

Output 1

```
Enter the number of days: 200

Equivalent Time:
Months         = 6
Remaining Days = 20
```

Output 2

```
Enter the number of days: 365

Equivalent Time:
Months         = 12
Remaining Days = 5
```

Output 3

```
Enter the number of days: 59

Equivalent Time:
Months         = 1
Remaining Days = 29
```

Practical No. 4

Total marks and percentage of six subjects

Aim

To write a C++ program to enter the marks of six subjects and find the Total Marks and Percentage.

Theory

Total Marks = sum of the marks of all six subjects and Percentage = Total Marks / 6. Each subject is assumed to be out of 100 marks.

Algorithm

1. Start the program.
2. Declare six variables for marks, and variables for total and percentage.
3. Accept the marks of all six subjects.
4. Calculate total = sum of all marks.
5. Calculate percentage = total / 6.0.
6. Display total marks and percentage.
7. Stop the program.

Program

```cpp
#include <iostream>
using namespace std;

int main()
{
    int sub1, sub2, sub3, sub4, sub5, sub6;
    int total;
    float percentage;

    cout << "Enter marks of Subject 1: ";
    cin >> sub1;
    cout << "Enter marks of Subject 2: ";
    cin >> sub2;
    cout << "Enter marks of Subject 3: ";
    cin >> sub3;
    cout << "Enter marks of Subject 4: ";
    cin >> sub4;
    cout << "Enter marks of Subject 5: ";
    cin >> sub5;
    cout << "Enter marks of Subject 6: ";
    cin >> sub6;

    total = sub1 + sub2 + sub3 + sub4 + sub5 + sub6;
    percentage = total / 6.0;

    cout << "\nTotal Marks = " << total;
    cout << "\nPercentage  = " << percentage << "%";
    return 0;
}
```

Output

```
Enter marks of Subject 1: 92
Enter marks of Subject 2: 81
Enter marks of Subject 3: 77
Enter marks of Subject 4: 89
Enter marks of Subject 5: 94
Enter marks of Subject 6: 70

Total Marks = 503
Percentage  = 83.8333%
```

Practical No. 5

~~Even or odd number~~

Aim

To write a C++ program to find whether the given number is even or odd.

Program

```cpp
#include <iostream>
using namespace std;

int main()
{
    int n;

    cout << "Enter a number: ";
    cin >> n;

    if (n % 2 == 0)
        cout << n << " is an Even number.";
    else
        cout << n << " is an Odd number.";
    return 0;
}
```

Output 1

```
Enter a number: 48
48 is an Even number.
```

Output 2

```
Enter a number: 73
73 is an Odd number.
```

Practical No. 6

Largest among three numbers

Aim

To write a C++ program to find the largest among three numbers.

Program

```cpp
#include <iostream>
using namespace std;

int main()
{
    int a, b, c;

    cout << "Enter three numbers: ";
    cin >> a >> b >> c;

    if (a >= b && a >= c)
        cout << "Largest number is: " << a;
    else if (b >= a && b >= c)
        cout << "Largest number is: " << b;
    else
        cout << "Largest number is: " << c;
    return 0;
}
```

Output 1

```
Enter three numbers: 34 87 52
Largest number is: 87
```

Output 2

```
Enter three numbers: 120 45 120
Largest number is: 120
```

Practical No. 7

Armstrong number

Aim

To write a C++ program to find whether the given number is an Armstrong number or not.

Program

```cpp
#include <iostream>
#include <cmath>
using namespace std;

int main()
{
    int num, originalNum, remainder, digits = 0;
    int result = 0;

    cout << "Enter a number: ";
    cin >> num;
    originalNum = num;

    // Count the number of digits
    for (; originalNum != 0; originalNum /= 10)
        digits++;

    originalNum = num;

    // Sum of digits raised to the power of number of digits
    for (; originalNum != 0; originalNum /= 10)
    {
        remainder = originalNum % 10;
        result += (int)round(pow(remainder, digits));
    }

    if (result == num)
        cout << num << " is an Armstrong Number.";
    else
        cout << num << " is not an Armstrong Number.";
    return 0;
}
```

Output 1

```
Enter a number: 9474
9474 is an Armstrong Number.
```

Output 2

```
Enter a number: 250
250 is not an Armstrong Number.
```

Practical No. 8

Table of a given number

Aim

To write a C++ program which generates the multiplication table of a given number.

Program

```cpp
#include <iostream>
using namespace std;

int main()
{
    int num;

    cout << "Enter a number: ";
    cin >> num;

    cout << "\nMultiplication Table of " << num << ":\n";
    for (int i = 1; i <= 10; i++)
        cout << num << " x " << i << " = " << num * i << endl;
    return 0;
}
```

Output

```
Enter a number: 13

Multiplication Table of 13:
13 x 1 = 13
13 x 2 = 26
13 x 3 = 39
13 x 4 = 52
13 x 5 = 65
13 x 6 = 78
13 x 7 = 91
13 x 8 = 104
13 x 9 = 117
13 x 10 = 130
```

Practical No. 9

~~Reference variable~~

Aim

To write a C++ program which uses the concept of a reference variable.

Program

```cpp
#include <iostream>
using namespace std;

int main()
{
    int num;

    cout << "Enter a number: ";
    cin >> num;

    int &ref = num;   // reference variable

    cout << "\nOriginal value of num = " << num << endl;
    cout << "Value using reference variable = " << ref << endl;

    ref = ref + 10;   // modify through the reference

    cout << "\nAfter modifying through reference variable:" << endl;
    cout << "Value of num = " << num << endl;
    cout << "Value of ref = " << ref << endl;
    return 0;
}
```

Output

```
Enter a number: 42

Original value of num = 42
Value using reference variable = 42

After modifying through reference variable:
Value of num = 52
Value of ref = 52
```

Practical No. 10

Swap two numbers using call by value

Aim

To write a C++ program to swap two numbers using the call by value method.

Program

```cpp
#include <iostream>
using namespace std;

// Call by value: the function works on copies of the arguments
void swapValues(int a, int b)
{
    int temp = a;
    a = b;
    b = temp;
    cout << "\nInside swapValues() function:" << endl;
    cout << "a = " << a << ", b = " << b << endl;
}

int main()
{
    int x, y;

    cout << "Enter two numbers: ";
    cin >> x >> y;

    cout << "\nBefore swapping:" << endl;
    cout << "x = " << x << ", y = " << y << endl;

    swapValues(x, y);

    cout << "\nAfter calling swapValues() function:" << endl;
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

Output

```
Enter two numbers: 33 77

Before swapping:
x = 33, y = 77

Inside swapValues() function:
a = 77, b = 33

After calling swapValues() function:
x = 33, y = 77
```

Practical No. 11

Swap two numbers using call by reference

Aim

To write a C++ program to swap two numbers using the call by reference method.

Program

```cpp
#include <iostream>
using namespace std;

// Call by reference: the function works on the original variables
void swapNumbers(int &a, int &b)
{
    int temp = a;
    a = b;
    b = temp;
}

int main()
{
    int num1, num2;

    cout << "Enter first number: ";
    cin >> num1;
    cout << "Enter second number: ";
    cin >> num2;

    cout << "\nBefore Swapping:" << endl;
    cout << "First Number  = " << num1 << endl;
    cout << "Second Number = " << num2 << endl;

    swapNumbers(num1, num2);

    cout << "\nAfter Swapping:" << endl;
    cout << "First Number  = " << num1 << endl;
    cout << "Second Number = " << num2 << endl;
    return 0;
}
```

Output

```
Enter first number: 8
Enter second number: 19

Before Swapping:
First Number  = 8
Second Number = 19

After Swapping:
First Number  = 19
Second Number = 8
```

Practical No. 12

Function with default arguments

Aim

To write a C++ program which uses a function with default arguments.

Program

```cpp
#include <iostream>
using namespace std;

// Function with default arguments
int add(int a = 15, int b = 25)
{
    return a + b;
}

int main()
{
    cout << "add()     = " << add() << endl;      // both defaults used
    cout << "add(7)    = " << add(7) << endl;     // a = 7, b = 25
    cout << "add(7, 3) = " << add(7, 3) << endl;  // no default used
    return 0;
}
```

Output

```
add()     = 40
add(7)    = 32
add(7, 3) = 10
```

Practical No. 13

Factorial using recursion

Aim

To write a C++ program to calculate the factorial of a number using recursion.

Program

```cpp
#include <iostream>
using namespace std;

// Recursive function to calculate factorial
int factorial(int n)
{
    if (n == 0 || n == 1)
        return 1;
    else
        return n * factorial(n - 1);
}

int main()
{
    int num;

    cout << "Enter a number: ";
    cin >> num;

    if (num < 0)
        cout << "Factorial of a negative number is not defined.";
    else
        cout << "Factorial of " << num << " = " << factorial(num);
    return 0;
}
```

Output 1

```
Enter a number: 8
Factorial of 8 = 40320
```

Output 2

```
Enter a number: -4
Factorial of a negative number is not defined.
```

Practical No. 14

Sum of array elements

Aim

To write a C++ program to take input in an array and print the sum of its elements.

Program

```cpp
#include <iostream>
using namespace std;

int main()
{
    int arr[100], n, sum = 0;

    cout << "Enter the number of elements: ";
    cin >> n;

    cout << "Enter " << n << " elements:" << endl;
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    for (int i = 0; i < n; i++)
        sum = sum + arr[i];

    cout << "Sum of array elements = " << sum << endl;
    return 0;
}
```

Output

```
Enter the number of elements: 6
Enter 6 elements:
12
7
25
33
4
19
Sum of array elements = 100
```

Practical No. 15

Count characters of a string

Aim

To write a C++ program to read a string and print the total number of characters in it.

Program

```cpp
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string str;

    cout << "Enter a string: ";
    getline(cin, str);

    cout << "Total number of characters = " << str.length() << endl;
    return 0;
}
```

Output

```
Enter a string: Object Oriented Programming
Total number of characters = 27
```

Additional Program (after Practical 15)

Simple program using class and object

Aim

To write a simple C++ program that uses a class and creates objects of that class (additional program).

Program

```cpp
#include <iostream>
#include <string>
using namespace std;

class Student
{
public:
    int rollNo;
    string name;

    void getData()
    {
        cout << "Enter Roll Number: ";
        cin >> rollNo;
        cout << "Enter Name: ";
        cin >> name;
    }

    void displayData()
    {
        cout << "\nStudent Details" << endl;
        cout << "Roll Number: " << rollNo << endl;
        cout << "Name: " << name << endl;
    }
};

int main()
{
    Student s1, s2;   // object creation

    s1.getData();
    s2.getData();

    s1.displayData();
    s2.displayData();
    return 0;
}
```

Output

```
Enter Roll Number: 407
Enter Name: Neel
Enter Roll Number: 412
Enter Name: Hetvi

Student Details
Roll Number: 407
Name: Neel
```

```
Student Details
Roll Number: 412
Name: Hetvi
```

Practical No. 16

Class Student with proper member functions

Aim

To write a C++ program to create a class called Student that has Name, ID, CPI, SPI and Semester as data members, with proper member functions for taking and displaying the information.

Program

```cpp
#include <iostream>
#include <string>
using namespace std;

class Student
{
private:
    string name;
    int id;
    float cpi, spi;
    int semester;

public:
    void getData()
    {
        cout << "Enter Student Name: ";
        getline(cin, name);
        cout << "Enter Student ID: ";
        cin >> id;
        cout << "Enter CPI: ";
        cin >> cpi;
        cout << "Enter SPI: ";
        cin >> spi;
        cout << "Enter Semester: ";
        cin >> semester;
    }

    void displayData()
    {
        cout << "\n----- Student Information -----" << endl;
        cout << "Name     : " << name << endl;
        cout << "ID       : " << id << endl;
        cout << "CPI      : " << cpi << endl;
        cout << "SPI      : " << spi << endl;
        cout << "Semester : " << semester << endl;
    }
};

int main()
{
    Student s;
    s.getData();
    s.displayData();
    return 0;
}
```

Output

```
Enter Student Name: Neel Parag Thakkar
Enter Student ID: 407
Enter CPI: 8.42
Enter SPI: 8.9
Enter Semester: 3

----- Student Information -----
Name     : Neel Parag Thakkar
ID       : 407
CPI      : 8.42
SPI      : 8.9
Semester : 3
```

Practical No. 17

Count objects using a static variable

Aim

To write a C++ program to count the number of objects created, using a static data member.

Program

```cpp
#include <iostream>
using namespace std;

class Student
{
private:
    static int count;   // static data member

public:
    Student()
    {
        count++;
        cout << "Object " << count << " created." << endl;
    }

    static void displayCount()
    {
        cout << "\nTotal number of objects created = " << count << endl;
    }
};

int Student::count = 0;   // definition of static data member

int main()
{
    Student s1, s2, s3, s4, s5;
    Student::displayCount();
    return 0;
}
```

Output

```
Object 1 created.
Object 2 created.
Object 3 created.
Object 4 created.
Object 5 created.

Total number of objects created = 5
```

Practical No. 18

Bank account using array of objects

Aim

To define a class representing a bank account (Name of depositor, Account number, Withdrawal amount, Balance) with member functions to assign initial values, deposit, withdraw after checking the balance, and display name and balance, and to handle n account holders using an array of objects.

Program

```cpp
#include <iostream>
#include <string>
using namespace std;

class BankAccount
{
private:
    string name;
    int accountNo;
    float balance;
    float withdrawAmount;

public:
    // assign initial values
    void assign()
    {
        cout << "\nEnter Depositor Name: ";
        cin.ignore();
        getline(cin, name);
        cout << "Enter Account Number: ";
        cin >> accountNo;
        cout << "Enter Initial Balance: ";
        cin >> balance;
    }

    // deposit an amount
    void deposit()
    {
        float amount;
        cout << "\nEnter amount to deposit: ";
        cin >> amount;
        balance = balance + amount;
        cout << "Amount Deposited Successfully." << endl;
    }

    // withdraw an amount after checking the balance
    void withdraw()
    {
        cout << "Enter amount to withdraw: ";
        cin >> withdrawAmount;
        if (withdrawAmount <= balance)
        {
            balance = balance - withdrawAmount;
            cout << "Withdrawal Successful." << endl;
        }
        else
        {
```

```
            cout << "Insufficient Balance!" << endl;
        }
    }

    // display name and balance
    void display()
    {
        cout << "\n-----------------------------" << endl;
        cout << "Depositor Name : " << name << endl;
        cout << "Account Number : " << accountNo << endl;
        cout << "Balance        : " << balance << endl;
    }
};

int main()
{
    int n;
    cout << "Enter Number of Account Holders: ";
    cin >> n;

    BankAccount *acc = new BankAccount[n];   // array of objects

    for (int i = 0; i < n; i++)
    {
        cout << "\nEnter Details of Account Holder " << i + 1 << endl;
        acc[i].assign();
    }

    for (int i = 0; i < n; i++)
    {
        cout << "\nTransactions for Account Holder " << i + 1 << endl;
        acc[i].deposit();
        acc[i].withdraw();
    }

    cout << "\n\n----- Account Details -----" << endl;
    for (int i = 0; i < n; i++)
        acc[i].display();

    delete[] acc;
    return 0;
}
```

Output

```
Enter Number of Account Holders: 3

Enter Details of Account Holder 1

Enter Depositor Name: Aarav Mehta
Enter Account Number: 1101
Enter Initial Balance: 7500

Enter Details of Account Holder 2

Enter Depositor Name: Riya Desai
Enter Account Number: 1102
Enter Initial Balance: 12000
```

```
Enter Details of Account Holder 3

Enter Depositor Name: Kunal Joshi
Enter Account Number: 1103
Enter Initial Balance: 3000

Transactions for Account Holder 1

Enter amount to deposit: 2500
Amount Deposited Successfully.
Enter amount to withdraw: 4000
Withdrawal Successful.

Transactions for Account Holder 2

Enter amount to deposit: 1000
Amount Deposited Successfully.
Enter amount to withdraw: 15000
Insufficient Balance!

Transactions for Account Holder 3

Enter amount to deposit: 500
Amount Deposited Successfully.
Enter amount to withdraw: 3200
Withdrawal Successful.


----- Account Details -----

-----------------------------
Depositor Name : Aarav Mehta
Account Number : 1101
Balance        : 6000

-----------------------------
Depositor Name : Riya Desai
Account Number : 1102
Balance        : 13000

-----------------------------
Depositor Name : Kunal Joshi
Account Number : 1103
Balance        : 300
```

Practical No. 19

~~Constructors and destructors~~

Aim

To write a C++ program to demonstrate the use of constructors and destructors.

Program

```cpp
#include <iostream>
using namespace std;

class Demo
{
    int id;

public:
    // Constructor
    Demo(int i)
    {
        id = i;
        cout << "Constructor called: object " << id << " created." << endl;
    }

    // Destructor
    ~Demo()
    {
        cout << "Destructor called: object " << id << " destroyed." << endl;
    }

    void display()
    {
        cout << "Hello from object " << id << " of the OOP lab!" << endl;
    }
};

int main()
{
    Demo d1(1);   // constructor is called automatically
    Demo d2(2);

    d1.display();
    d2.display();
    return 0;     // destructors are called automatically
}
```

Output

```
Constructor called: object 1 created.
Constructor called: object 2 created.
Hello from object 1 of the OOP lab!
Hello from object 2 of the OOP lab!
Destructor called: object 2 destroyed.
Destructor called: object 1 destroyed.
```

Practical No. 20

Copy constructor for complex numbers

Aim

To write a C++ program which reads a complex number and copies it into another object using a copy constructor.

Program

```cpp
#include <iostream>
using namespace std;

class Complex
{
    float real, imag;

public:
    // Parameterized constructor
    Complex(float r = 0, float i = 0)
    {
        real = r;
        imag = i;
    }

    // Copy constructor
    Complex(const Complex &c)
    {
        real = c.real;
        imag = c.imag;
    }

    void display()
    {
        cout << real;
        if (imag >= 0)
            cout << " + " << imag << "i";
        else
            cout << " - " << -imag << "i";
        cout << endl;
    }
};

int main()
{
    float real, imag;

    cout << "Enter real part: ";
    cin >> real;
    cout << "Enter imaginary part: ";
    cin >> imag;

    Complex c1(real, imag);
    Complex c2(c1);   // copy c1 into c2 using the copy constructor

    cout << "\nOriginal Complex Number: ";
    c1.display();
    cout << "Copied Complex Number  : ";
    c2.display();
```

```
50      return 0;
51  }
```

Output

```
Enter real part: 3.5
Enter imaginary part: -2.5

Original Complex Number: 3.5 - 2.5i
Copied Complex Number  : 3.5 - 2.5i
```

Practical No. 21

Function overloading for geometric area

Aim

To write a C++ program implementing function overloading to find the area of different geometric figures.

Program

```cpp
#include <iostream>
using namespace std;

// Area of circle
float area(float radius)
{
    return 3.14159 * radius * radius;
}

// Area of rectangle
float area(float length, float breadth)
{
    return length * breadth;
}

// Area of triangle (extra int parameter only to make the signature unique)
float area(float base, float height, int)
{
    return 0.5 * base * height;
}

int main()
{
    float r, l, b, base, h;

    cout << "Enter radius of circle: ";
    cin >> r;
    cout << "Area of Circle = " << area(r) << endl;

    cout << "\nEnter length and breadth of rectangle: ";
    cin >> l >> b;
    cout << "Area of Rectangle = " << area(l, b) << endl;

    cout << "\nEnter base and height of triangle: ";
    cin >> base >> h;
    cout << "Area of Triangle = " << area(base, h, 0) << endl;
    return 0;
}
```

Output

```
Enter radius of circle: 7
Area of Circle = 153.938

Enter length and breadth of rectangle: 12 4.5
Area of Rectangle = 54

Enter base and height of triangle: 9 6
Area of Triangle = 27
```

Practical No. 22

Addition of complex numbers using friend function

Aim

To write a C++ program for addition of two complex numbers using a friend function (a constructor is used to initialize the data members of the Complex class).

Program

```cpp
#include <iostream>
using namespace std;

class Complex
{
    float real, imag;

public:
    // Constructor to initialize data members
    Complex(float r = 0, float i = 0)
    {
        real = r;
        imag = i;
    }

    // Friend function declaration
    friend Complex add(Complex, Complex);

    void display()
    {
        cout << real << " + " << imag << "i";
    }
};

// Friend function definition
Complex add(Complex c1, Complex c2)
{
    Complex c3;
    c3.real = c1.real + c2.real;
    c3.imag = c1.imag + c2.imag;
    return c3;
}

int main()
{
    float r1, i1, r2, i2;

    cout << "Enter real and imaginary part of first complex number: ";
    cin >> r1 >> i1;
    cout << "Enter real and imaginary part of second complex number: ";
    cin >> r2 >> i2;

    Complex c1(r1, i1);
    Complex c2(r2, i2);
    Complex c3 = add(c1, c2);

    cout << "\nFirst Complex Number  = ";
    c1.display();
    cout << "\nSecond Complex Number = ";
```

```
50      c2.display();
51      cout << "\nAddition              = ";
52      c3.display();
53      return 0;
54  }
```

Output

```
Enter real and imaginary part of first complex number: 6 2
Enter real and imaginary part of second complex number: 3 9

First Complex Number  = 6 + 2i
Second Complex Number = 3 + 9i
Addition              = 9 + 11i
```

Practical No. 23

Mean value using friend function

Aim

To write a C++ program to find the mean value of given numbers using a friend function.

Program

```cpp
#include <iostream>
using namespace std;

class Mean
{
    int n;
    float arr[100];

public:
    Mean(int count)
    {
        n = count;
    }

    void getData()
    {
        cout << "Enter " << n << " numbers: ";
        for (int i = 0; i < n; i++)
            cin >> arr[i];
    }

    friend float calculateMean(Mean);
};

float calculateMean(Mean m)
{
    float sum = 0;
    for (int i = 0; i < m.n; i++)
        sum += m.arr[i];
    return sum / m.n;
}

int main()
{
    int n;

    cout << "Enter number of values: ";
    cin >> n;

    Mean m(n);
    m.getData();

    cout << "Mean = " << calculateMean(m);
    return 0;
}
```

Output

```
Enter number of values: 6
Enter 6 numbers: 12 18 25 31 9 15
Mean = 18.3333
```

Practical No. 24

Add integer values of two class objects

Aim

To write a C++ program to add the integer values of two class objects by overloading the + operator.

Program

```cpp
#include <iostream>
using namespace std;

class Number
{
    int value;

public:
    // Constructor
    Number(int v = 0)
    {
        value = v;
    }

    // Operator overloading for +
    Number operator+(Number obj)
    {
        Number temp;
        temp.value = value + obj.value;
        return temp;
    }

    void display()
    {
        cout << "Sum = " << value << endl;
    }
};

int main()
{
    Number n1(45), n2(27), n3;

    n3 = n1 + n2;
    n3.display();
    return 0;
}
```

Output

```
Sum = 72
```

Practical No. 25

Addition of two time objects

Aim

To write a C++ program to add two time objects (hours, minutes and seconds) by overloading the + operator.

Program

```cpp
#include <iostream>
using namespace std;

class Time
{
    int hours, minutes, seconds;

public:
    void getTime()
    {
        cout << "Enter hours, minutes and seconds: ";
        cin >> hours >> minutes >> seconds;
    }

    void display()
    {
        cout << hours << " hours "
             << minutes << " minutes "
             << seconds << " seconds";
    }

    // Operator overloading for +
    Time operator+(Time t)
    {
        Time temp;
        temp.seconds = seconds + t.seconds;
        temp.minutes = minutes + t.minutes + temp.seconds / 60;
        temp.seconds = temp.seconds % 60;
        temp.hours = hours + t.hours + temp.minutes / 60;
        temp.minutes = temp.minutes % 60;
        return temp;
    }
};

int main()
{
    Time t1, t2, t3;

    cout << "Enter first time:\n";
    t1.getTime();
    cout << "Enter second time:\n";
    t2.getTime();

    t3 = t1 + t2;

    cout << "\nFirst Time  = ";
    t1.display();
    cout << "\nSecond Time = ";
    t2.display();
```

```
50      cout << "\nAddition of Two Times = ";
51      t3.display();
52      return 0;
53  }
```

Output

```
Enter first time:
Enter hours, minutes and seconds: 2 50 45
Enter second time:
Enter hours, minutes and seconds: 4 20 30

First Time  = 2 hours 50 minutes 45 seconds
Second Time = 4 hours 20 minutes 30 seconds
Addition of Two Times = 7 hours 11 minutes 15 seconds
```

Practical No. 26

Static member function showcount()

Aim

To write a C++ program to design a class having a static member function named showcount() which displays the number of objects created of the class.

Program

```cpp
#include <iostream>
using namespace std;

class Demo
{
    static int count;

public:
    Demo()
    {
        count++;
    }

    static void showcount()
    {
        cout << "Number of objects created = " << count << endl;
    }
};

int Demo::count = 0;   // definition of static data member

int main()
{
    Demo d1, d2, d3;
    Demo::showcount();

    Demo d4, d5;
    Demo::showcount();
    return 0;
}
```

Output

```
Number of objects created = 3
Number of objects created = 5
```

Practical No. 27

Unary operator overloading (++ / –)

Aim

To write a C++ program which uses the concept of unary operator (++ / –) overloading.

Program

```cpp
#include <iostream>
using namespace std;

class Number
{
    int value;

public:
    Number(int v)
    {
        value = v;
    }

    // Unary ++ operator overloading
    void operator++()
    {
        ++value;
    }

    // Unary -- operator overloading
    void operator--()
    {
        --value;
    }

    void display()
    {
        cout << "Value = " << value << endl;
    }
};

int main()
{
    Number n(25);

    cout << "Initial value:" << endl;
    n.display();

    ++n;
    cout << "After increment:" << endl;
    n.display();

    --n;
    --n;
    cout << "After two decrements:" << endl;
    n.display();
    return 0;
}
```

Output

```
Initial value:
Value = 25
After increment:
Value = 26
After two decrements:
Value = 24
```

Practical No. 28

Binary operator overloading on strings

Aim

To write a C++ program which uses the concept of binary operator (+) overloading on strings.

Program

```cpp
#include <iostream>
#include <cstring>
using namespace std;

class String
{
    char str[100];

public:
    String(const char s[] = "")
    {
        strcpy(str, s);
    }

    // Binary + operator overloading (concatenation)
    String operator+(String s)
    {
        String temp;
        strcpy(temp.str, str);
        strcat(temp.str, s.str);
        return temp;
    }

    void display()
    {
        cout << str << endl;
    }
};

int main()
{
    String s1("Electronics ");
    String s2("and Communication");
    String s3;

    s3 = s1 + s2;

    cout << "First String        : ";
    s1.display();
    cout << "Second String       : ";
    s2.display();
    cout << "Concatenated String : ";
    s3.display();
    return 0;
}
```

Output

```
First String        : Electronics
Second String       : and Communication
Concatenated String : Electronics and Communication
```

Practical No. 29

Matrix addition using operator overloading

Aim

To write a C++ program to create a Matrix class and overload the + operator to add two matrix objects.

Program

```cpp
#include <iostream>
using namespace std;

class Matrix
{
    int a[2][2];

public:
    void getdata()
    {
        cout << "Enter matrix elements:" << endl;
        for (int i = 0; i < 2; i++)
            for (int j = 0; j < 2; j++)
                cin >> a[i][j];
    }

    // Binary + operator overloading
    Matrix operator+(Matrix m)
    {
        Matrix temp;
        for (int i = 0; i < 2; i++)
            for (int j = 0; j < 2; j++)
                temp.a[i][j] = a[i][j] + m.a[i][j];
        return temp;
    }

    void display()
    {
        for (int i = 0; i < 2; i++)
        {
            for (int j = 0; j < 2; j++)
                cout << a[i][j] << "\t";
            cout << endl;
        }
    }
};

int main()
{
    Matrix m1, m2, m3;

    cout << "Enter elements of first matrix:" << endl;
    m1.getdata();
    cout << "Enter elements of second matrix:" << endl;
    m2.getdata();

    m3 = m1 + m2;

    cout << "\nFirst Matrix:" << endl;
```

```
    m1.display();
    cout << "\nSecond Matrix:" << endl;
    m2.display();
    cout << "\nResultant Matrix:" << endl;
    m3.display();
    return 0;
}
```

Output

```
Enter elements of first matrix:
Enter matrix elements:
4 7 2 9
Enter elements of second matrix:
Enter matrix elements:
3 1 8 5

First Matrix:
4       7
2       9

Second Matrix:
3       1
8       5

Resultant Matrix:
7       8
10      14
```

Practical No. 30

Polar and rectangular coordinates conversion

Aim

To define two classes Polar and Rectangle representing a point in the polar and rectangular systems, and use conversion routines to convert from one system to the other.

Program

```
#include <iostream>
#include <cmath>
using namespace std;

class Polar;

class Rectangle
{
    float x, y;

public:
    Rectangle(float a = 0, float b = 0)
    {
        x = a;
        y = b;
    }

    Rectangle(Polar p);   // conversion constructor (Polar -> Rectangle)

    friend class Polar;

    void display()
    {
        cout << "Rectangular Coordinates = ("
             << x << ", " << y << ")" << endl;
    }
};

class Polar
{
    float r, theta;

public:
    Polar(float radius = 0, float angle = 0)
    {
        r = radius;
        theta = angle;
    }

    Polar(Rectangle rec)   // conversion constructor (Rectangle -> Polar)
    {
        r = sqrt(rec.x * rec.x + rec.y * rec.y);
        theta = atan2(rec.y, rec.x) * 180 / 3.14159;
    }

    friend class Rectangle;

    void display()
    {
```

```
        cout << "Polar Coordinates = ("
             << r << ", " << theta << " degrees)" << endl;
    }
};

Rectangle::Rectangle(Polar p)
{
    float angle = p.theta * 3.14159 / 180;
    x = p.r * cos(angle);
    y = p.r * sin(angle);
}

int main()
{
    Rectangle R1(5, 12);
    cout << "Original Rectangle:\n";
    R1.display();

    Polar P1 = R1;   // Rectangle -> Polar
    cout << "\nRectangle to Polar:\n";
    P1.display();

    Rectangle R2 = P1;   // Polar -> Rectangle
    cout << "\nPolar to Rectangle:\n";
    R2.display();
    return 0;
}
```

Output

```
Original Rectangle:
Rectangular Coordinates = (5, 12)

Rectangle to Polar:
Polar Coordinates = (13, 67.3802 degrees)

Polar to Rectangle:
Rectangular Coordinates = (5, 12)
```

Practical No. 31

Single inheritance

Aim

To write a C++ program to implement single inheritance.

Program

```cpp
#include <iostream>
#include <string>
using namespace std;

// Base class
class Student
{
protected:
    int rollNo;
    string name;

public:
    void getStudentData()
    {
        cout << "Enter Roll Number: ";
        cin >> rollNo;
        cout << "Enter Name: ";
        cin >> name;
    }

    void displayStudentData()
    {
        cout << "Roll Number: " << rollNo << endl;
        cout << "Name: " << name << endl;
    }
};

// Derived class
class Result : public Student
{
private:
    float marks;

public:
    void getResult()
    {
        cout << "Enter Marks: ";
        cin >> marks;
    }

    void displayResult()
    {
        displayStudentData();
        cout << "Marks: " << marks << endl;
    }
};

int main()
{
    Result r;
```

```
51
52      r.getStudentData();
53      r.getResult();
54
55      cout << "\n--- Student Result ---\n";
56      r.displayResult();
57      return 0;
58  }
```

Output

```
Enter Roll Number: 407
Enter Name: Neel
Enter Marks: 88.5

--- Student Result ---
Roll Number: 407
Name: Neel
Marks: 88.5
```

Practical No. 32

Multiple inheritance

Aim

To write a C++ program to implement multiple inheritance.

Program

```cpp
#include <iostream>
#include <string>
using namespace std;

// Base class 1
class Student
{
protected:
    int rollNo;
    string name;

public:
    void getStudentData()
    {
        cout << "Enter Roll Number: ";
        cin >> rollNo;
        cout << "Enter Name: ";
        cin >> name;
    }
};

// Base class 2
class Marks
{
protected:
    float marks;

public:
    void getMarks()
    {
        cout << "Enter Marks: ";
        cin >> marks;
    }
};

// Derived class
class Result : public Student, public Marks
{
public:
    void display()
    {
        cout << "\n--- Student Result ---" << endl;
        cout << "Roll Number: " << rollNo << endl;
        cout << "Name: " << name << endl;
        cout << "Marks: " << marks << endl;
    }
};

int main()
{
```

```
    Result r;

    r.getStudentData();
    r.getMarks();
    r.display();
    return 0;
}
```

Output

```
Enter Roll Number: 412
Enter Name: Hetvi
Enter Marks: 91

--- Student Result ---
Roll Number: 412
Name: Hetvi
Marks: 91
```

Practical No. 33

File handling using a constructor function

Aim

To write a C++ program to create a file using a constructor function (the file is opened in the constructor and closed in the destructor).

Program

```cpp
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

class FileDemo
{
    ofstream fout;

public:
    // Constructor creates (opens) the file
    FileDemo(const char *fileName)
    {
        fout.open(fileName);
        cout << "File \"" << fileName << "\" created." << endl;
    }

    void writeLine(const string &text)
    {
        fout << text << endl;
    }

    // Destructor closes the file
    ~FileDemo()
    {
        fout.close();
        cout << "File closed." << endl;
    }
};

int main()
{
    string name, id;

    cout << "Enter your name: ";
    getline(cin, name);
    cout << "Enter your ID: ";
    getline(cin, id);

    {
        FileDemo f("student.txt");
        f.writeLine("Name : " + name);
        f.writeLine("ID   : " + id);
    }

    // Read the file back and display its contents
    ifstream fin("student.txt");
    string line;
    cout << "\nContents of student.txt:" << endl;
```

```
50      while (getline(fin, line))
51          cout << line << endl;
52      fin.close();
53      return 0;
54  }
```

Output

```
Enter your name: Neel Parag Thakkar
Enter your ID: 25EC407
File "student.txt" created.
File closed.

Contents of student.txt:
Name : Neel Parag Thakkar
ID   : 25EC407
```

Practical No. 34

Use of template

Aim

To write a C++ program to demonstrate the use of a template.

Program

```cpp
#include <iostream>
using namespace std;

// Function template
template <class T>
T largest(T a, T b)
{
    return (a > b) ? a : b;
}

// Class template
template <class T>
class Pair
{
    T first, second;

public:
    Pair(T a, T b)
    {
        first = a;
        second = b;
    }

    T sum()
    {
        return first + second;
    }
};

int main()
{
    cout << "Largest of 17 and 42       : " << largest(17, 42) << endl;
    cout << "Largest of 3.75 and 2.5    : " << largest(3.75, 2.5) << endl;
    cout << "Largest of 'k' and 'c'     : " << largest('k', 'c') << endl;

    Pair<int> p1(14, 28);
    Pair<float> p2(1.5f, 2.25f);
    cout << "Sum of integer pair        : " << p1.sum() << endl;
    cout << "Sum of float pair          : " << p2.sum() << endl;
    return 0;
}
```

Output

```
Largest of 17 and 42       : 42
Largest of 3.75 and 2.5    : 3.75
Largest of 'k' and 'c'     : k
Sum of integer pair        : 42
Sum of float pair          : 3.75
```